

GAME PROGRAMMING COURSE — C# BASICS

C# 復習

変数・入出力・条件分岐・配列・関数・構造体

対象：1年生（復習）

REVIEW

W

LESSON 1

変数・入出力

目標：変数の使い方と、入力・出力の書き方を思い出す



このコマの目標

- 変数の宣言・型・代入を思い出す
- Console.WriteLine による出力の書き方を思い出す
- Console.ReadLine による入力を受け取り方を思い出す
- 入力した文字列を数値に変換して使えるようになる

変数の復習

- 変数＝データに名前をつけて入れておく箱。型（int, string, bool など）ごとに扱えるデータが決まっている。

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         int age = 15;
9         string name = " ゆうた ";
10        bool isActive = true;
11        Console.WriteLine(name + " さんは " + age + " 歳です ");
12    }
13 }
```

入力: Console.ReadLine()

- Console.ReadLine() で、キーボードから入力された 1 行分の文字列を受け取れる。

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         Console.WriteLine("名前を入力してください");
9         string input = Console.ReadLine();
10        Console.WriteLine("こんにちは、 " + input + " さん");
11    }
12 }
```

型変換して使う

- Console.ReadLine() の結果は必ず string。数値として計算したいときは int.Parse などに変換する。

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         Console.WriteLine(" 数字を入力してください ");
9         string input = Console.ReadLine();
10        int number = int.Parse(input);
11        Console.WriteLine(number + 10);
12    }
13 }
```

このコマのポイント

- 変数は「型」を決めて、名前をつけてデータを入れておく箱
- `Console.WriteLine(...)` で画面に出力する
- `Console.ReadLine()` で入力を文字列として受け取る
- `int.Parse(...)` など、文字列を数値に変換してから計算する

Paiza でよく使う入力①: スペース区切りの文字

- 1 行の中に複数の文字列がスペースで区切られている場合、`Split(' ')` で分割してから使う。

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         string line = Console.ReadLine();
9         string[] words = line.Split(' ');
10        Console.WriteLine(words[0]);
11        Console.WriteLine(words[1]);
12    }
13 }
```


Paiza でよく使う入力②: スペース区切りの数値

- 分割した文字列を `int.Parse` で数値に変換する。 `Select(int.Parse)` を使うとまとめて変換できる。

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         string line = Console.ReadLine();
9         int[] numbers = line.Split(' ').Select(int.Parse).ToArray();
10        Console.WriteLine(numbers[0] + numbers[1]);
11    }
12 }
```

Paiza でよく使う入力③: 1行1つの数値

- 1行に数値が1つだけ書かれている場合は、ReadLine で読んでそのまま int.Parse するだけでよい。

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         string line = Console.ReadLine();
9         int number = int.Parse(line);
10        Console.WriteLine(number);
11    }
12 }
```

LESSON 2

条件分岐・配列

目標: if ・ switch と、配列・多次元配列の使い方を思い出す



このコマの目標

- if 文・ switch 文の書き方を思い出す
- for 文・ while 文の書き方を思い出す
- 配列の宣言・添字によるアクセスを思い出す
- 多次元配列（2次元配列）の使い方を思い出す

if 文の復習

C# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         int hp = 15;
9         if (hp <= 0)
10         {
11             Console.WriteLine(" ゲームオーバー ");
12         }
13         else if (hp < 20)
14         {
15             Console.WriteLine(" 危険! ");
16         }
17         else
18         {
19             Console.WriteLine(" 元気です ");
20         }
21     }
22 }
```

switch 文の復習

- 1つの値に対して選択肢が多いときは、switch を使うと見やすくなる。

c# Program.cs

```
1  int day = 3;
2  switch (day)
3  {
4      case 1:
5          Console.WriteLine(" 月曜日 ");
6          break;
7      case 2:
8          Console.WriteLine(" 火曜日 ");
9          break;
10     default:
11         Console.WriteLine(" その他の曜日 ");
12         break;
13 }
```

for 文の復習

- 「初期化; 条件; 増減」をまとめて書き、決まった回数だけ繰り返す処理に向く。

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         for (int i = 0; i < 5; i++)
9         {
10             Console.WriteLine(i);
11         }
12     }
13 }
```

while 文の復習

- 条件が真である間、繰り返し続ける。繰り返す回数が決まっていないときに向く。

c# Program.cs

```

1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         int i = 0;
9         while (i < 5)
10        {
11            Console.WriteLine(i);
12            i++;
13        }
14    }
15 }

```

POINT

for は「繰り返す回数が決まっている」とき、while は「条件次第で回数が変わる」ときに使い分けると分かりやすい。

配列の復習

C# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         int[] scores = { 80, 65, 90 };
9         Console.WriteLine(scores[0]);
10        scores[1] = 70;
11        for (int i = 0; i < scores.Length; i++)
12        {
13            Console.WriteLine(scores[i]);
14        }
15    }
16 }
```

多次元配列の復習

- `int[,]` のように書くと、縦・横を持つ 2 次元配列を扱える。

c# Program.cs

```
1 int[,] board = new int[2, 3];
2 board[0, 0] = 1;
3 board[1, 2] = 5;
4 for (int y = 0; y < 2; y++)
5 {
6     for (int x = 0; x < 3; x++)
7     {
8         Console.Write(board[y, x] + " ");
9     }
10    Console.WriteLine();
11 }
```

foreach 文の復習

- 配列の添字を意識せず、中身を 1 つずつ順番に取り出したいときに使う。

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 class Program
5 {
6     static void Main()
7     {
8         int[] scores = { 80, 65, 90 };
9         foreach (int score in scores)
10        {
11            Console.WriteLine(score);
12        }
13    }
14 }
```

POINT

for は「添字」を使って読み書きしたいとき、foreach は「中身を順番に見るだけ」でよいときに向く。

このコマのポイント

- `if / else if / else` で、条件に応じた処理を分けられる
- `switch` は、1つの値に対する選択肢が多いときに見やすい
- `for` は決まった回数、`while` は条件次第の回数を繰り返すときに使う
- `foreach` は配列の中身を、添字を使わずに1つずつ取り出せる
- 配列は `型 [] 変数名 = { 値, 値, ... };` で宣言し、添字 `[ 番号 ]` で読み書きする
- 多次元配列は `型 [,] 変数名 = new 型 [ 縦, 横 ];` で宣言する

演習へ： Paiza の問題を一緒に解く



2つの数値を出力 C# 編

https://paiza.jp/works/mondai/d_rank_level_up_problems/d_rank_level_up_problems_stdout_boss/edit?language_uid=c-sharp



数値演算結果で分岐 C# 編

https://paiza.jp/works/mondai/conditions_branch/conditions_branch_simple_step3?language_uid=c-sharp



足したり引いたり C# 編

https://paiza.jp/works/mondai/conditions_branch/conditions_branch_complex_step2?language_uid=c-sharp



0が含まれていないか判定 C# 編

https://paiza.jp/works/mondai/conditions_branch/conditions_branch_simple_boss/edit?language_uid=c-sharp



割り切れる数だけ出力 C# 編

https://paiza.jp/works/mondai/loop_problems2/loop_problems2_div_output?language_uid=c-sharp



荷物検査 C# 編

https://paiza.jp/works/mondai/rank_test_problems_c_0001/rank_test_problems_c_0001_3/edit?language_uid=c-sharp

演習へ： Paiza の問題を一緒に解く（続き）



宝くじ C# 編

https://paiza.jp/works/mondai/c_rank_skillcheck_archive/lottery/edit?language_uid=c-sharp



みかんの仕分け C# 編

https://paiza.jp/works/mondai/c_rank_skillcheck_archive/mikan/edit?language_uid=c-sharp

LESSON 3

関数

目標：関数の定義・引数・戻り値の使い方を思い出す



このコマの目標

- 関数（メソッド）とは何かを思い出す
- 引数と戻り値の使い方を思い出す
- 配列を関数に渡したり、関数から受け取ったりできるようになる

関数とは（復習）

- まとまった処理に名前をつけ、何度でも呼び出せるようにしたもの
- 戻り値がある場合は int などの型を、ない場合は void を書く

C# Program.cs

```
1 static int Add(int a, int b)
2 {
3     return a + b;
4 }
5
6 // 呼び出す側
7 int result = Add(3, 5);
8 Console.WriteLine(result);
```

引数と戻り値

- 引数＝関数に渡す値。戻り値がない関数（void）は、return を書かなくてよい。

c# Program.cs

```
1 static void ShowResult(string name, int score)
2 {
3     Console.WriteLine(name + "さんの得点は" + score);
4 }
5
6 // 呼び出す側
7 ShowResult("ゆうた", 90);
```

配列を関数でやり取りする

C# Program.cs

```
1 static int Sum(int[] numbers)
2 {
3     int total = 0;
4     foreach (int n in numbers)
5     {
6         total += n;
7     }
8     return total;
9 }
10
11 // 呼び出す側
12 int[] scores = { 80, 65, 90 };
13 Console.WriteLine(Sum(scores));
```

このコマのポイント

- 関数は「まとまった処理」に名前をつけて再利用できるようにする仕組み
- 戻り値があるときは型を、ないときは void を書く
- 引数として配列を渡し、foreach などの中身で処理できる
- 同じ処理をあちこちに書かず、関数にまとめることでコードが整理される

LESSON 4

構造体・総合演習

目標：構造体を復習し、これまでの内容を組み合わせて使えるようになる



このコマの目標

- 構造体の定義・インスタンス化を思い出す
- 構造体の配列の使い方を思い出す
- クラスの書き方を軽く思い出す
- 変数・入出力・条件分岐・配列・関数・構造体を組み合わせた問題に取り組む

構造体の復習

c# Program.cs

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 public struct PlayerData
5 {
6     public string Name;
7     public int Hp;
8 }
9 class Program
10 {
11     static void Main()
12     {
13         PlayerData p1 = new PlayerData();
14         p1.Name = "ゆうた";
15         p1.Hp = 100;
16         Console.WriteLine(p1.Name + " " + p1.Hp);
17     }
18 }
```

構造体の配列（復習）

c# Program.cs

```
1  PlayerData[] players = new PlayerData[2];
2  players[0].Name = " ゆうた ";
3  players[0].Hp = 100;
4  players[1].Name = " みさき ";
5  players[1].Hp = 80;
6
7  foreach (PlayerData p in players)
8  {
9      Console.WriteLine(p.Name + " " + p.Hp);
10 }
```


クラスの軽い復習

c# Program.cs

```
1 public class PlayerData
2 {
3     public string Name;
4     public int Hp;
5
6     public void TakeDamage(int dmg)
7     {
8         Hp -= dmg;
9     }
10 }
11 class Program
12 {
13     static void Main()
14     {
15         PlayerData p1 = new PlayerData();
16         p1.Name = "ゆうた ";
17         p1.Hp = 100;
18         p1.TakeDamage(20);
19         Console.WriteLine(p1.Name + " " + p1.Hp);
20     }
21 }
```

総合演習の紹介

- お題：入力を受け取り、複数のキャラクターを構造体で管理するプログラム
- ① Console.ReadLine で人数やキャラクター情報を受け取る
- ② 構造体の配列に読み取った情報を格納する
- ③ 条件分岐や関数を使って、状態の判定やダメージ計算などを行う
- これまでの 4 コマの内容をすべて使う総合問題です

演習へ： Paiza の問題を解く



構造体の更新 C# 編

https://paiza.jp/works/mondai/class_primer/class_primer_change/edit?language_uid=c-sharp



文字と整数の組のソート C# 編

https://paiza.jp/works/mondai/prob60/sort_asc_9/edit?language_uid=c-sharp



文字と整数の組のソート 2 C# 編

https://paiza.jp/works/mondai/prob60/sort_add_9/edit?language_uid=c-sharp

全体のまとめ

- 変数・入出力: データを箱に入れ、入力を受け取り、画面に出力する
- 条件分岐・配列: if/switch で処理を分け、配列・多次元配列でデータをまとめて扱う
- 関数: 処理に名前をつけて再利用し、引数・戻り値でデータをやり取りする
- 構造体: 複数のデータを 1 つの型としてまとめ、配列としても扱える

お疲れさまでした

ここまでの基礎はすべて、この先のクラスや
Unity でのゲーム制作を支える土台になります。